Al-Furqan — Codebase Knowledge Graph Documentation

Complete Architecture & Component Reference

Project: Al-Furqan (الفرقان) — Axiom-Anchored Reasoning Engine **Generated:** March 21, 2026 **Git Commit:** `39a0664` **Tool:** Understand-Anything Knowledge Graph Generator

---

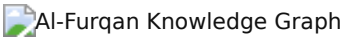
1. Project Overview

Al-Furqan is a neuro-symbolic reasoning engine that evaluates ideas, systems, policies, and claims against immutable axioms through formal verification. It sits as an evaluation layer on top of any LLM.

Metric	Value
Files analyzed	52
Total nodes	511 (52 files, 349 functions, 110 classes)
Total edges	554 (459 contains, 83 imports, 12 tested_by)
Architectural layers	9
Languages	Python
Frameworks	FastAPI, ChromaDB, Pydantic, Pytest

---

2. Knowledge Graph Visualization



File-level architecture showing 9 layers, import dependencies (solid arrows), and test relationships (dashed arrows).

---

3. Architectural Layers

3.1 Core Reasoning Layer (5 files)

The brain of the system — pure reasoning logic.

File	Purpose
`reasoning_engine.py`	Constitutional core — axioms, gates, prompt templates, evaluation pipeline
`cot.py`	Chain-of-Thought data structures and result models
`cot_engine.py`	CoT execution engine — guided reasoning chains per gate
`cot_prompts.py`	CoT prompt templates for structured extraction

`__init__.py`	Module exports

Key classes: `ReasoningEngine`, `Verdict`, `GateScore`, `GateResult`, `SystemType`, `DualPerspectiveVerdict`, `CoTEngine`

Design principle: LLM extracts, Code scores, Z3 proves.

3.2 API Layer (11 files)

FastAPI application — the external interface.

File	Purpose
`app.py`	FastAPI application factory, CORS, middleware registration
`schemas.py`	Pydantic request/response models
`dependencies.py`	Dependency injection (engine, store, config)
`converters.py`	Internal → API response format converters
`routers/evaluate.py`	POST /evaluate — main evaluation endpoint
`routers/criterion.py`	POST /criterion — criterion-based evaluation
`routers/verdicts.py`	GET/DELETE /verdicts — verdict management
`routers/review.py`	Review workflow endpoints
`routers/stats.py`	GET /stats — system statistics
`routers/__init__.py`	Router registration
`__init__.py`	Module exports

Endpoints: `/api/v1/evaluate`, `/api/v1/criterion`, `/api/v1/verdicts`, `/api/v1/stats`, `/api/v1/review`

3.3 Authentication & Security Layer (8 files)

API key management, middleware, rate limiting.

File	Purpose
`key_manager.py`	API key CRUD — create, validate, revoke, list
`models.py`	APIKey, UserRole, Permission data models
`middleware.py`	Authentication middleware — validates Bearer tokens
`security.py`	Password hashing (bcrypt), input sanitization
`rate_limiter.py`	Token bucket rate limiting per API key
`errors.py`	Custom auth exception classes
`cli.py`	CLI tool for key management

`__init__.py`	Module exports

Roles: Admin, Evaluator, Reader — hierarchical permissions.

3.4 LLM Provider Layer (2 files)

Multi-provider LLM abstraction.

File	Purpose
`llm_layer.py`	Unified interface for Claude, Qwen, Ollama, OpenAI — model routing, retry logic, streaming
`__init__.py`	Module exports

Supported providers: Anthropic (Claude), OpenAI, Ollama (local), Qwen.

3.5 Storage Layer (2 files)

Verdict persistence with ChromaDB vector search.

File	Purpose
`verdict_store.py`	Store/retrieve/search verdicts — JSON + ChromaDB vectors
`__init__.py`	Module exports

Features: Full-text search, semantic similarity search, verdict invalidation, statistics.

3.6 Human Review Layer (2 files)

Interactive verdict review workflow.

File	Purpose
`human_review.py`	Interactive CLI for reviewing and approving/correcting verdicts
`__init__.py`	Module exports

3.7 CLI & Configuration Layer (3 files)

Entry points and configuration management.

File	Purpose
`cli.py`	Main CLI entry point — evaluate, review, manage
`config.py`	YAML-based configuration — models, providers, thresholds
`__main__.py`	Python module entry point

3.8 Scripts & Tooling Layer (5 files)

Development and maintenance scripts.

File	Purpose
`batch_test.py`	Run batch evaluations for testing
`render_pdf.py`	Render Markdown + Mermaid diagrams to PDF
`render_pdf_v2.py`	Improved PDF renderer
`vectorize_verdicts.py`	Re-vectorize existing verdicts in ChromaDB
`celery_app.py`	Celery worker for async evaluation tasks

3.9 Test Layer (14 files)

Comprehensive pytest test suite.

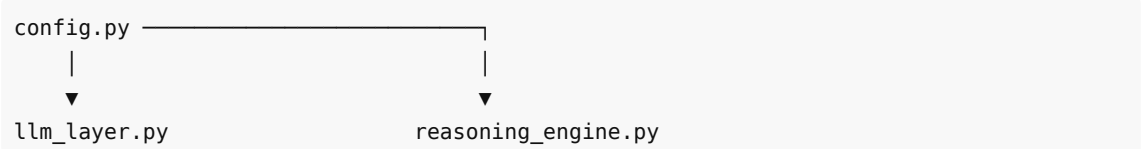
File	Tests
`test_reasoning_engine.py`	Core evaluation pipeline, gates, scoring
`test_cot.py`	Chain-of-Thought engine and prompts
`test_verdict_store.py`	Storage CRUD, search, statistics
`test_llm_layer.py`	LLM provider routing, retry, fallback
`test_config.py`	Configuration loading, validation
`test_auth.py`	Authentication middleware
`test_security.py`	Password hashing, input sanitization
`test_rate_limiter.py`	Rate limiting logic
`test_human_review.py`	Review workflow
`test_api_evaluate.py`	Evaluation endpoint integration
`test_api_health.py`	Health check endpoint
`test_api_verdicts.py`	Verdict management endpoints
`conftest.py`	Shared fixtures (engine, store, mocks)
`test_dual_perspective.py`	Dual-perspective evaluation

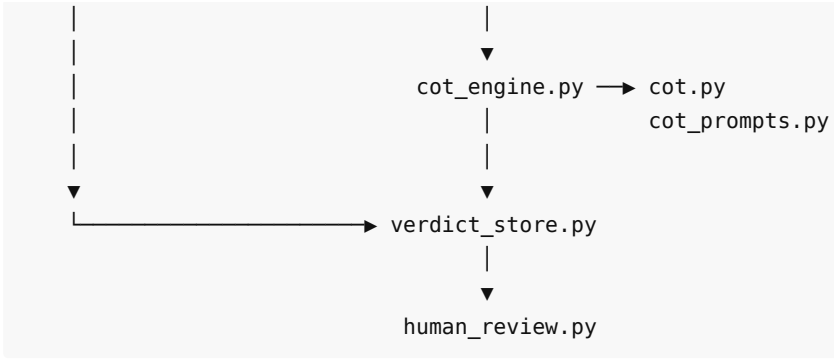
Coverage: 205+ tests passing.

---

4. Dependency Map (Import Graph)

4.1 Core Dependencies





4.2 API Layer Dependencies

```
app.py
  ├──▶ config.py
  ├──▶ llm_layer.py
  ├──▶ reasoning_engine.py
  ├──▶ verdict_store.py
  ├──▶ key_manager.py
  ├──▶ middleware.py
  ├──▶ security.py
  └──▶ errors.py

dependencies.py
  ├──▶ config.py
  ├──▶ reasoning_engine.py
  └──▶ verdict_store.py

routers/*.py
  ├──▶ dependencies.py
  ├──▶ schemas.py
  ├──▶ converters.py
  └──▶ (specific stores/engines as needed)
```

4.3 Test Dependencies

```
conftest.py (shared fixtures)
  ├──▶ reasoning_engine.py (mock LLM)
  ├──▶ verdict_store.py (temp storage)
  ├──▶ llm_layer.py (provider mocks)
  ├──▶ key_manager.py (test keys)
  └──▶ models.py (test roles)

test_*.py → conftest.py (fixtures)
          → src module under test
```

---

5. Guided Learning Tour

A 12-step tour for new developers joining the project:

Step 1: Project Entry Points

📁 `cli.py` , `__main__.py`

> *Start here to understand how the application boots. The CLI provides commands for evaluation, review, and key management.*

Step 2: Configuration

📁 `config.py`

> *YAML-based configuration defines LLM providers, model names, scoring thresholds, and storage paths. Everything is configurable without code changes.*

Step 3: LLM Provider Abstraction

📁 `llm_layer.py`

> *The unified interface to all LLM providers. Handles model routing, retry logic, and streaming. Swap providers without touching the engine.*

Step 4: The Reasoning Engine (Core)

📁 `reasoning_engine.py`

> ***The heart of Al-Furqan.*** *Contains immutable axioms, gate definitions, prompt templates, and the evaluation pipeline (Scan → Mirror → Verdict → Self-Correction). This is the constitutional core.*

Step 5: Chain-of-Thought Engine

📁 `cot_engine.py` , `cot.py` , `cot_prompts.py`

> *Guided reasoning chains — the LLM answers structured questions per gate, extracting facts that code then scores deterministically.*

Step 6: Verdict Storage

📁 `verdict_store.py`

> *Persists evaluation results as JSON + ChromaDB vectors. Supports full-text search, semantic similarity, and statistical analysis.*

Step 7: Human Review

📁 `human_review.py`

> *Interactive CLI workflow for humans to review, approve, or correct verdicts. Critical for quality assurance and feedback loops.*

Step 8: API Schemas

📁 `schemas.py` , `converters.py`

> *Pydantic models defining the API contract. Converters transform internal data structures to API response formats.*

Step 9: API Routers

📁 `routers/evaluate.py` , `routers/criterion.py` , `routers/verdicts.py` , `routers/stats.py` , `routers/review.py`

> *FastAPI route handlers. Each router owns a domain: evaluation, verdict management, statistics, review workflow.*

Step 10: Authentication & Security

📁 `key_manager.py` , `middleware.py` , `security.py` , `rate_limiter.py`

> *API key management, request authentication, password hashing, input sanitization, and rate limiting. Multi-role permission system.*

Step 11: Knowledge Base Scripts

📁 `scripts/vectorize_verdicts.py` , `scripts/batch_test.py`

> *Development tools for re-vectorizing verdicts and running batch evaluations.*

Step 12: Test Suite

📁 `tests/`

> *205+ tests covering every layer. Start with `test_reasoning_engine.py` to understand the core, then explore outward.*

---

6. Key Design Patterns

6.1 LLM as Tongue, Not Brain

The LLM speaks (extraction, explanation). The Code thinks (scoring, computation). Z3 proves (formal verification). The Knowledge Base knows (verified sources).

6.2 Separation of Concerns

Engine ↔ Knowledge ↔ Storage — layers never cross-import. Only the Orchestrator (API layer) connects them.

6.3 Deterministic Scoring

Same inputs → same score, regardless of LLM model. The LLM extracts structured data; scoring functions are pure Python with no LLM involvement.

6.4 Self-Correction Loop

After initial evaluation, the engine checks for contradictions in its own reasoning and re-runs affected phases up to 3 times.

---

7. Statistics

Node Distribution

Type	Count	%

Functions	349	68.3%
Classes	110	21.5%
Files	52	10.2%
Total	**511**	**100%**

Edge Distribution

Type	Count	%
Contains (file→function/class)	459	82.9%
Imports (file→file)	83	15.0%
Tested By (src→test)	12	2.2%
Total	**554**	**100%**

Layer Sizes

Layer	Files	Functions	Classes
API Layer	11	~80	~25
Test Layer	14	~100	~15
Auth & Security	8	~60	~20
Core Reasoning	5	~50	~30
Scripts & Tooling	5	~30	~10
CLI & Config	3	~15	~5
LLM Provider	2	~10	~3
Storage	2	~8	~2
Human Review	2	~5	~2

Generated by Understand-Anything · Al-Furqan Codebase Analysis Full knowledge graph data: `.understand-anything/knowledge-graph.json`